

VoltEZ

Gap Analysis Report

Errors & Shortcomings vs. Initial Plan

Branch: final-frontend | Generated: August 28, 2026

Severity Legend

CRITICAL	Blocks demo or causes data loss
HIGH	Missing core functionality planned for this sprint
MEDIUM	Degraded experience or significant technical debt
LOW	Polish, consistency, or future-proofing issue

1 EXECUTIVE SUMMARY

VoltEZ is an EV charging discovery, reservation, and decision-intelligence platform built for a hackathon. The project spans a Flutter mobile app (driver + business sides), a FastAPI modular-monolith backend, PostgreSQL/PostGIS database, Redis-based booking locks, and an ML pipeline with four core models.

This report audits the current state of the codebase on the 'final-frontend' branch against the documented architecture, API contract, database schema, and planned features. It identifies 30+ distinct gaps organized by severity across six domains: Infrastructure, Backend API, Frontend/Flutter, ML Integration, Database, and Testing/Quality.

Key Findings

1. Infrastructure: docker-compose only defines PostgreSQL. Redis and Celery (required for booking holds and background jobs) are missing from the container stack.
2. Backend API: The API contract defines ~35 endpoints. Approximately 10 critical endpoints are unimplemented or return placeholders (e.g., routes/recommendations, business analytics, booking list for owners).
3. Frontend (Driver Side): All 11 driver screens use 100% hardcoded mock data. None make live API calls. The LiveBookingApi class throws UnimplementedError everywhere.
4. Frontend (Business Side): Only the dashboard exists as a real screen. The charger management, availability scheduler, bookings, analytics, profile, and onboarding screens are all empty placeholder directories.
5. ML Integration: The backend's MLAdapter has hardcoded fallback heuristics and no actual model loading. The trained models (demand, availability) sit in 'artifacts/' but are not wired into any backend endpoint.
6. State Management Conflict: The project uses both 'provider' (ChangeNotifier) and 'flutter_riverpod' simultaneously, which violates the stated conventions.
7. Testing: Backend has only confest.py and an empty integration/ directory. No unit tests, no API tests. The frontend has 2 test files (widget_test.dart, dashboard_test.dart) covering almost nothing.

2 INFRASTRUCTURE

2.1 Redis Not in docker-compose

CRITICAL

The booking system requires Redis for atomic slot locking (SET NX with TTL). The backend booking.py calls `redis.set(lock_key, ...)` and `redis.enqueue_job()`. docker-compose.yml only defines the PostgreSQL container. Without Redis, the entire booking flow crashes on every attempt.

Planned (architecture.md): Redis 7 for booking slot locks, caching, and Celery broker.

Actual: docker-compose.yml has only the 'db' service (postgres/postgis:17-3.5). No Redis service is defined. The .env.example lists REDIS_URL but it has no backing container.

2.2 Celery / Background Worker Not Deployed

CRITICAL

The architecture specifies Celery for background jobs (booking expiry, notifications, settlement). A worker.py exists in backend/app/ but there is no Celery worker container, no task queue configuration, and no worker process in docker-compose.

Planned: Celery workers for expiry cleanup, notification dispatch, settlement batching.

Actual: backend/app/worker.py exists but is not wired to any process manager or container orchestration. Redis broker is also missing.

2.3 No Redis Service Configuration

HIGH

The .env.example template references REDIS_URL but provides no default port mapping. Even if Redis were added to docker-compose, the backend would need explicit connection configuration that currently only exists as a placeholder.

The backend app's Redis connection setup (likely in app/db/session.py or similar) should be audited for proper failover and connection pooling.

2.4 Missing Container for Frontend

MEDIUM

The Flutter app has no containerization. While mobile apps are typically deployed directly, the web target mentioned in the architecture (Next.js Business Web) has no corresponding container or build step.

Planned: Next.js Business Web for browser-based owner dashboard.

Actual: Only a Flutter web target exists. No separate web dashboard build.

3 BACKEND API - MISSING & INCOMPLETE ENDPOINTS

3.1 Routes / Recommendations Endpoint

CRITICAL POST /routes/recommendations is the core differentiator for the driver experience. The frontend defines getRouteRecommendations() in api_service.dart and the API contract specifies it, but the backend's recommendations.py returns a placeholder JSON {"message": "Business intelligence engine is under construction."} for the business variant, and the driver variant calls recommendation_service which is not implemented with actual ML logic.

Frontend method: ApiService.getRouteRecommendations()

Contract: POST /api/v1/routes/recommendations

Actual: The endpoint exists but returns static placeholder data or raises errors.

Impact: Route planner, recommendations screen, and booking flow all depend on this.

3.2 Business Analytics Endpoint

HIGH GET /businesses/{id}/analytics is listed in the API contract but the backend's analytics.py router exists without a fully implemented aggregation pipeline. The frontend BusinessNotifier calls /businesses/me/analytics/revenue which has no corresponding backend route.

Planned: Real-time revenue, utilization, and session analytics per business.

Actual: The analytics router is registered but the revenue aggregation query is not implemented. The frontend will receive 404 errors.

3.3 Owner-Filtered Booking List

HIGH GET /bookings (for owners to see bookings for their chargers) is not implemented. The frontend business side needs this to populate the Bookings tab. The existing booking.py only supports driver-side booking creation and cancellation.

Planned: Owners see all bookings for their chargers with filtering.

Actual: No owner-specific booking list endpoint exists. The GET /bookings route returns driver bookings only.

3.4 PATCH /chargers/{id} - Charger Update

HIGH The API contract specifies PATCH /chargers/{id} for editing charger details. The frontend's ApiService.updateCharger() exists but the backend's charger.py only has POST / (create), GET /nearby, and POST /{id}/report-issue. No PATCH route.

Frontend method: ApiService.updateCharger(int id, Map data)

Actual: 404 - endpoint does not exist in the backend.

3.5 DELETE /chargers/{id} - Charger Deletion

MEDIUM The frontend ApiService.deleteCharger() calls DELETE /chargers/{id} but the backend charger.py has no DELETE route. Only the businesses router has a delete endpoint.

Frontend method: ApiService.deleteCharger(int id)

Actual: 404 - no route registered.

3.6 Business Registration & Profile Route Mismatch

MEDIUM The frontend BusinessNotifier calls /businesses/me and /businesses/me/chargers. The backend has GET /businesses/ (list by owner) and POST /businesses/ (create), but no /businesses/me shorthand. The frontend will need to adapt or the backend needs a /me alias.

Planned: Seamless business profile management.

Actual: Route mismatch - frontend uses /me, backend uses /{business_id}.

3.7 Pricing Recommendations

MEDIUM BusinessNotifier.fetchPricingRecommendations() calls GET /pricing/recommendations and acceptPricingRecommendation() calls POST /pricing/recommendations/{id}/accept. No /pricing router exists in the backend.

Planned: ML-driven dynamic pricing recommendations for business owners.
Actual: No pricing endpoint exists. The ML adapter has no pricing model.

3.8 AI Copilot Endpoint

LOW BusinessNotifier.queryAiCopilot() calls POST /businesses/me/copilot-query. No such endpoint exists in the backend. This is likely a Gemini/Lyze integration placeholder from the architecture spec.

Planned: AI-powered business insights via Gemini/Lyze.
Actual: No endpoint, no integration.

3.9 System Health Endpoints

LOW GET /health/live, GET /health/ready, and GET /version are in the API contract but no corresponding routes are registered in the backend.

Planned: Liveness/readiness probes for container orchestration.
Actual: Not implemented.

4 FRONTEND (FLUTTER) - GAPS

4.1 All Driver Screens Use Mock Data

CRITICAL Every single driver-side screen (11 screens) uses hardcoded mock data. None make actual API calls to the backend. The LiveBookingApi class throws UnimplementedError() for every method. The driver home screen hardcodes battery percent (72%), charger names, and addresses as Dart literals.

Screens affected: DriverHomeScreen, DriverMapScreen, RoutePlannerInputScreen, RoutePlannerScreen, ChargerDetailsScreen, BookingScreen, PaymentScreen, BookingConfirmationScreen, ChargingSessionScreen, DriverHistoryScreen, DriverOnboardingScreen.

The mock adapters are clearly named (MockBookingApi, MockSessionApi, etc.) which is good practice, but the switch to live backends has not been executed.

4.2 Business Side: Missing Core Screens

CRITICAL Of 7 planned business screens, only DashboardScreen exists as a real implementation. The remaining 6 directories are EMPTY:

- bookings/ (0 files)
- profile/ (0 files)
- analytics/ (0 files)
- onboarding/ (0 files)

The charger_management_screen.dart and port_details_screen.dart exist but use mock data. The availability_scheduler_screen.dart exists but uses mock data.

Planned: Full business owner dashboard with charger CRUD, availability scheduling, booking management, analytics, profile, and onboarding.

Actual: Dashboard + mock charger list + empty placeholders.

4.3 Router Uses Placeholder Screens

HIGH The app_router.dart routes for /business/chargers, /business/availability, /business/bookings, /business/analytics, and /business/profile all point to a _PlaceholderScreen widget that shows 'Coming soon' text. These should be wired to the real screen implementations.

The placeholder class _PlaceholderScreen is defined inline in app_router.dart and renders a simple 'Coming soon' message for 5 out of 6 business routes.

4.4 State Management Conflict: Provider + Riverpod

HIGH The project simultaneously uses two state management solutions:

1. 'provider' (ChangeNotifier) - used by main.dart, AuthProvider, SessionProvider, BookingProvider, ChargerDiscoveryProvider, RoutePlannerProvider.
2. 'flutter_riverpod' - used by business_provider.dart, api_client.dart.

The pubspec.yaml lists both 'provider: ^6.1.5' and 'flutter_riverpod: ^2.6.1'. The project conventions (BUSINESS_DEVELOPER_GUIDE.md) state 'provider (ChangeNotifier)' as the chosen state management solution.

This creates confusion about which pattern to follow, makes dependency injection inconsistent, and will cause issues when providers need to cross-reference each other.

4.5 Dashboard Uses Custom Color Constants, Not AppColors

MEDIUM The DashboardScreen defines its own color constants (_ivory, _ink, _forest, _rust, _fadeInk, _panel) instead of using the shared AppColors from lib/core/theme/colors.dart. The driver side correctly uses AppColors everywhere. This

creates a visual inconsistency and violates the design system.

DashboardScreen colors: `_ivory=#05090E`, `_forest=#50F5FF`, `_rust=#C9FF58`

AppColors: `background=#0A0F1F`, `primary=#00E5FF`, `success=#34D399`

The dashboard's palette is notably different from the shared design system.

4.6 Business Dashboard Is 100% Hardcoded Mock Data

HIGH The DashboardScreen hardcodes everything: business name 'ABC Motors', stats (8 active chargers, 24 bookings, Rs.18.4K revenue, 76% utilization), charger list, booking list, utilization chart bars, and the AI insight text. None of this is fetched from any provider or API.

The MockBusinessApi adapter exists but is never used by the DashboardScreen. The screen builds all widgets directly from Dart literal values.

4.7 Missing Business Provider Registration in main.dart

HIGH main.dart registers providers for Auth, ChargerDiscovery, RoutePlanner, Booking, and Session. The BusinessNotifier (from business_provider.dart using Riverpod) is NOT registered anywhere. Any screen using businessProvider will fail at runtime.

main.dart providers: AuthProvider, ChargerDiscoveryProvider, RoutePlannerProvider, BookingProvider, SessionProvider.

Missing: BusinessNotifier/BusinessProvider.

4.8 Legacy Screens Directory

LOW The lib/screens/ directory contains empty subdirectories for splash, bookings, auth, shell, business, explore, dashboard, profile, earnings, and notifications. The BUSINESS_DEVELOPER_GUIDE.md says 'lib/screens/ is legacy - don't add new files there.' These empty directories should be cleaned up to avoid confusion.

Empty directories found in lib/screens/: splash/, bookings/, auth/, shell/, business/, explore/, dashboard/, profile/, earnings/, notifications/.

5 ML INTEGRATION - GAPS

5.1 Models Not Loaded into Backend

CRITICAL The trained models exist as joblib bundles in `models/demand/voltex-demand-60m-pune-v1/` and `models/availability/voltex-availability-pune-v1/`. The backend's MLAdapter (`backend/app/ml/adapters.py`) accepts a 'model' parameter but defaults to None, falling back to hardcoded heuristics (`expected_demand=1.0`, `confidence=0.50`). No model loading code exists in the backend startup path.

The ML serving layer in `src/voltex_ml/serving/` has proper prediction functions (`demand.py`, `availability.py`) but the backend does not import or use them.

Impact: All ML-powered features (demand forecasting, availability prediction) return dummy values in production.

5.2 MLAdapter Feature Building Is Incomplete

HIGH The MLAdapter calls `build_demand_features(charger_id)` and `build_availability_features(charger_id, port_id)` from `app/services/ml_features.py`. These functions likely build feature vectors from the database, but the feature specifications in the ML pipeline (52 features for demand, 35 for availability) require complex point-in-time joins that are not implemented in the backend.

The ML training pipeline builds features from synthetic data using `src/voltex_ml/features/suite.py`. The backend would need equivalent real-time feature extraction from PostgreSQL, which is a significant engineering effort not yet attempted.

5.3 No Model Serving Contract in Backend Startup

HIGH The `docs/backend_fastapi_handoff.md` specifies exactly how models should be loaded at FastAPI startup. The backend's `main.py` does not reference any model loading, hash verification, or feature contract validation at startup.

Planned: Load models at startup, verify SHA-256 hashes, validate feature contracts, fail fast if artifacts are corrupted.

Actual: No model loading in backend startup.

5.4 Wait-Time Prediction Logic Is Placeholder

MEDIUM The `MLAdapter.predict_wait_time()` method translates probability of unavailability to wait minutes via a naive formula: `'wait_minutes = prob_unavailable * 60.0'`. The actual ML pipeline has a dedicated Model 3 (waiting time) with its own training and serving code, but the backend does not use it.

Planned: Model 3 (waiting time) prediction using trained model.

Actual: Simplistic probability-to-minutes conversion.

5.5 Reliability Model (Model 4) Not Integrated

MEDIUM Model 4 (reliability prediction) has training code in `src/voltex_ml/training/reliability.py` and serving code in `src/voltex_ml/serving/reliability.py`. The backend trust service uses a simple `+5/-20` point heuristic instead of the trained model.

Planned: ML-based reliability prediction from trained Model 4.

Actual: Simple additive heuristic in TrustService.

5.6 Route-Energy Model (Model 5) Not Integrated

MEDIUM Model 5 (route-energy) has physics-based code in `src/voltex_ml/route_energy/` but no backend integration. The route recommendation endpoint should use it for reachability and energy estimation.

Planned: Physics-based route energy estimation with vehicle profiles.

Actual: Code exists in ML package but is not called from any backend endpoint.

5.7 No ML Prediction Audit Trail in Database

MEDIUM The database schema defines ml_lab.ml_predictions for audit logging. The backend's MLAdapter.log_prediction() writes to this table, but since the models are not loaded, all predictions logged will be dummy values with hardcoded confidence scores.

The audit trail mechanism exists but will only record fallback heuristic results.

6 DATABASE SCHEMA - GAPS

6.1 Zone Table Not Used

MEDIUM

The schema defines app.zones for geographic demand aggregation. The database/models/ directory has zone.py, but the backend API does not expose any zone-related endpoints. Chargers are not associated with zones in the current implementation.

Planned: Every charger belongs to a zone for demand forecasting.

Actual: Zones exist as a model but are not used in any API flow.

6.2 Charging Requests Table Not Populated

MEDIUM

app.charging_requests is the primary source for Model 1's demand label. The ChargerSearchEvent model logs searches, but charging_requests (which should capture route planning requests, nearby searches, and scheduled searches) is not populated by any API endpoint.

Planned: Every driver interaction (search, route plan, booking attempt) populates charging_requests for demand measurement.

Actual: Only ChargerSearchEvent is logged, which is a subset of the planned instrumentation.

6.3 Parking Spaces Not Implemented

LOW

app.parking_spaces and the parking_space_id field on bookings are defined in the schema but not implemented in the backend models or API.

This is a lower-priority feature that can be deferred.

6.4 Host Settlements Table Not Implemented

LOW

app.host_settlements for aggregating host earnings is defined in the schema but not implemented. The business analytics feature depends on this.

Planned: Automated settlement calculation for business owners.

Actual: Not implemented.

6.5 Audit Events Table Not Implemented

LOW

app.audit_events for security and administrative audit trails is defined in the schema but no corresponding model or service exists in the backend.

This is a compliance/security feature that can be deferred for a hackathon.

7 TESTING & QUALITY - GAPS

7.1 Backend Has Zero Unit Tests

CRITICAL The backend/tests/ directory contains only confest.py and an empty integration/ subdirectory. There are zero test files for any API endpoint, service, repository, or schema. The booking state machine, charger CRUD, session lifecycle, and payment flow are completely untested.

Planned: pytest + pytest-asyncio for unit and integration tests.

Actual: Empty test directory.

7.2 Frontend Has Minimal Test Coverage

HIGH The test/ directory contains widget_test.dart (likely a default Flutter counter test) and dashboard_test.dart. No tests exist for any driver screen, provider, API service, router, or auth flow.

With 11 driver screens and a complex booking/session flow, the absence of widget and integration tests means regressions will be caught only by manual testing.

7.3 No E2E or Integration Tests

HIGH There are no integration tests that verify the full booking flow (hold -> payment -> confirm -> check-in -> start -> complete). This is the most critical user journey and has zero automated verification.

The test_driver/ directory does not exist. No Flutter integration tests exist.

7.4 No CI/CD Pipeline

MEDIUM No GitHub Actions, no CI configuration, no linting pipeline. The project has analysis_options.yaml and pyproject.toml with ruff/mypy configured, but no automated checks run on push or PR.

Planned: Automated quality gates.

Actual: No CI/CD configuration files found.

7.5 ML Pipeline Has Good Test Coverage (Exception)

LOW The ML pipeline (tests/ in project root) has 22 test files covering synthetic generation, features, training, and serving. These are thorough and well-designed. However, the backend application code has zero corresponding tests.

This is a positive finding - the ML side is well-tested. The gap is specifically in the backend application and frontend Flutter tests.

8 SECURITY & CORRECTNESS CONCERNS

8.1 Demo Login Bypasses All Authentication

HIGH AuthProvider.demoLogin() creates a fake user with id=9999 without any backend call. This is clearly labeled for demo/testing, but if accessible in production builds, it allows full access without authentication.

The demoLogin method should be behind a debug flag or removed from release builds.

8.2 Booking Flow Missing Backend Verification

HIGH The frontend booking flow (MockBookingApi) creates local hold states without backend verification. When switching to live mode, the hold->payment->confirm sequence must be server-validated. The current mock pattern does not test for this critical security property.

The backend booking.py does implement Redis locking, but the frontend mock never actually calls the backend.

8.3 Token Refresh Not Implemented

MEDIUM The ApiService has refreshTokens() method but the _AuthInterceptor does not implement automatic token refresh on 401 responses. The AuthProvider stores refreshToken but never uses it proactively.

The interceptor should catch 401 errors, attempt a token refresh, and retry the original request. Currently, any expired token causes a permanent logout.

8.4 Error Interceptor Wrapping Issue

MEDIUM The _ErrorInterceptor in api_service.dart wraps DioException with ApiError but the error type is attached to the 'error' field, not the 'response'. Callers checking err.response.data will still see raw backend data instead of the parsed ApiError.

The error wrapping pattern needs review to ensure consistent error handling across the app.

8.5 IDs Inconsistently Typed (String vs UUID vs int)

MEDIUM The backend uses UUID primary keys, but the frontend's charger ID in DriverHomeScreen is a String ('c1'). The GoRouter path uses state.pathParameters['id'] as String. The ApiService.getChargerById() takes int. This type mismatch will cause runtime errors when connecting to the real backend.

Backend: UUID (string)

Frontend router: String path parameters

ApiService: int parameter

This inconsistency will cause failures in production.

9 DOCUMENTATION INCONSISTENCIES

9.1 API Contract vs. Actual Backend Endpoints

The docs/api-contract.md lists endpoints that don't exist in the backend:

- POST /auth/verify (OTP verification)
- GET /businesses/{id}/analytics (analytics aggregation)
- GET /businesses/{id}/recommendations (AI recommendations)
- POST /chargers/{id}/ports (add port to charger)
- PATCH /ports/{id} (update port)
- POST /routes/quote (detailed quote)
- POST /payments/webhook (provider callback)
- POST /payments/{id}/refund (refund)
- GET /ml/predictions/{entity_id}
- GET /ml/health
- GET /health/live, /health/ready, /version

9.2 Frontend-API Mapping Document Shows Stale Status

docs/frontend-api-mapping.md was auto-generated and shows some endpoints as 'stub' or 'not yet implemented' that may now have partial implementations, while not reflecting new endpoints added in the backend.

9.3 BUSINESS_DEVELOPER_GUIDE Lists Non-Existent Endpoints

The guide says 'These endpoints are defined in the backend schemas but NOT yet implemented as API routes' for Business endpoints. Some have since been implemented (POST /businesses, GET /businesses, PATCH /businesses/{id}) but the guide still labels them as missing.

9.4 Architecture Doc Mentions Next.js Business Web

docs/architecture.md shows 'Next.js Business Web' as a client. This was replaced by the Flutter business side. The architecture diagram should be updated.

10 SEVERITY SUMMARY

Severity	Area	Issue
CRITICAL	Infrastructure	Redis missing from docker-compose
CRITICAL	Infrastructure	Celery worker not deployed
CRITICAL	Backend API	Routes/recommendations endpoint unimplemented
CRITICAL	Frontend	All 11 driver screens use 100% mock data
CRITICAL	Frontend	6 of 7 business screens are empty directories
CRITICAL	ML Integration	Trained models not loaded into backend
CRITICAL	Testing	Backend has zero unit tests
HIGH	Backend API	Business analytics endpoint unimplemented
HIGH	Backend API	Owner booking list endpoint missing
HIGH	Backend API	PATCH /chargers/{id} not implemented
HIGH	Frontend	Router uses placeholder screens for 5 routes
HIGH	Frontend	State management conflict (Provider + Riverpod)
HIGH	Frontend	Dashboard 100% hardcoded, not using providers
HIGH	Frontend	Business provider not registered in main.dart
HIGH	ML Integration	Feature building not implemented in backend
HIGH	ML Integration	No model serving contract at startup
HIGH	Testing	Frontend has minimal test coverage
HIGH	Testing	No E2E or integration tests
HIGH	Testing	Backend tests directory empty
HIGH	Security	Demo login bypasses authentication
HIGH	Security	Booking flow not verified through backend
MEDIUM	Backend API	DELETE /chargers/{id} missing
MEDIUM	Backend API	Business /me route mismatch
MEDIUM	Backend API	Pricing recommendations endpoint missing
MEDIUM	Frontend	Dashboard uses custom colors, not AppColors
MEDIUM	ML Integration	Wait-time prediction is naive placeholder
MEDIUM	ML Integration	Reliability model uses simple heuristic
MEDIUM	ML Integration	Route-energy model not integrated
MEDIUM	ML Integration	Prediction audit trail records dummy values
MEDIUM	Database	Zones not used in any API flow
MEDIUM	Database	Charging requests table not populated
MEDIUM	Security	Token refresh not implemented
MEDIUM	Security	Error interceptor swallows information
MEDIUM	Security	ID type inconsistency (String vs UUID vs int)
MEDIUM	Testing	No CI/CD pipeline
LOW	Backend API	AI Copilot endpoint missing
LOW	Backend API	Health/version endpoints missing
LOW	Frontend	Legacy screens/ directory has empty dirs

LOW	Database	Parking spaces not implemented
LOW	Database	Host settlements not implemented
LOW	Database	Audit events not implemented

11 PRIORITIZED RECOMMENDATIONS

Immediate (Before Next Demo)

1. Add Redis to docker-compose.yml with proper volume and port mapping.
2. Add Celery worker container or at least a process manager for background jobs.
3. Wire at least one driver screen (e.g., charger discovery) to real API calls.
4. Implement the missing PATCH /chargers/{id} endpoint.
5. Fix the ID type mismatch between frontend (String/int) and backend (UUID).

Short-Term (Within 1 Week)

6. Standardize on one state management solution (recommend: Provider as documented).
7. Migrate DashboardScreen to use AppColors and real data from providers.
8. Implement the business analytics aggregation endpoint.
9. Wire the ML models into backend startup with hash verification.
10. Write basic pytest tests for booking, auth, and charger CRUD flows.

Medium-Term (Before Production)

11. Implement token refresh interceptor for automatic 401 handling.
12. Build the remaining business screens (bookings, profile, analytics).
13. Switch all driver screens from mock to live adapters.
14. Implement real-time feature building for ML predictions.
15. Add CI/CD pipeline with lint, typecheck, and test gates.

Long-Term (Post-Hackathon)

16. Implement notification system (Celery + FCM).
17. Add parking space management.
18. Implement host settlement calculations.
19. Build the audit events system.
20. Add WebSocket-based real-time updates for sessions.